

VIGIL PROJECT BRIEF · INTERNAL DOCUMENT

VIGIL

Cashew Quality Inspection System

Intern Project Brief · Sprint: June 19 – 23, 2026

Document Version 2.0 · Classification: Internal / Intern Use

This document is the sole reference for all intern contributors assigned to the VIGIL MVP sprint. It defines what you are building, why it matters, how to build it, and the exact AI agent prompts you should use when coding. Read every section relevant to your role before writing a single line of code. There is no separate onboarding session — this document replaces it.

PROJECT CODE	VIGIL-MVP-01	DURATION	5 days (19–23 Jun 2026)
TEAM SIZE	3 Interns	REVIEW	Daily — 6:00 PM IST
DELIVERABLE	Working demo video + full codebase on GitHub	STACK	Python 3.10, OpenCV, anomalib, FastAPI, SQLite

NON-DISCLOSURE AGREEMENT

CONFIDENTIALITY & NON-DISCLOSURE NOTICE

This document and all information contained herein — including but not limited to system architecture, source code, AI model design, dataset methodology, inference pipeline, and any derivative materials — is strictly confidential and is issued exclusively to the named intern contributor for the purpose of completing the VIGIL MVP sprint (June 19–23, 2026). Receipt of this document constitutes acceptance of all terms stated below.

1. PROHIBITION ON DISCLOSURE

The recipient must not share, distribute, transmit, publish, or otherwise disclose the contents of this document — in whole or in part — to any individual, organisation, or entity not directly authorised by the project owner. This includes sharing via messaging applications, email, cloud storage, social media, academic submissions, or any other medium, whether digital or physical.

2. PROHIBITION ON REVERSE ENGINEERING

The recipient must not attempt to reverse-engineer, decompile, reconstruct, replicate, or repurpose any component of the VIGIL system — including the model architecture, inference pipeline, dataset collection methodology, backend API design, or dashboard implementation — for use in any personal, commercial, academic, or third-party project without express written authorisation.

3. PENALTIES FOR BREACH

Any breach of this agreement — including unauthorised disclosure or reverse engineering — will result in immediate legal action. The offending individual will be liable for a civil fine of RUPEES TEN LAKHS (Rs. 10,00,000) per individual found to be in breach, recoverable through applicable civil courts. In addition, criminal proceedings may be initiated under applicable Indian law, which may result in imprisonment of up to FIVE (5) YEARS.

4. SCOPE & DURATION

These obligations apply from the date of receipt of this document and remain in force indefinitely, including after the conclusion of the internship. The end of the sprint period does not release the recipient from any obligation stated herein.

5. GOVERNING LAW

This agreement is governed by the laws of India. Any dispute arising from a breach of this agreement shall be subject to the exclusive jurisdiction of the competent courts of India.

RECIPIENT ACKNOWLEDGEMENT

By receiving and reading this document, the recipient confirms they have read, understood, and agree to be bound by all terms stated above. Receipt constitutes acceptance.

Full Name: _____

Intern Role: _____

Date Received: _____

SECTION 01

Project Overview & Objectives

VIGIL is a real-time, offline, AI-powered visual quality inspection system designed for cashew processing facilities. It detects defective kernels on a production line using a standard USB or IP camera and an anomaly detection model running entirely on CPU — no internet connection, no GPU, no cloud dependency.

WHY THIS APPROACH

Traditional defect detection systems require thousands of labelled defect images. VIGIL uses EfficientAD, an unsupervised anomaly detection model that trains only on good-quality kernels. It learns what 'normal' looks like, and flags anything that deviates from that baseline as a potential defect. This eliminates the labelling problem entirely and makes the system deployable in facilities with no ML expertise.

SUCCESS DEFINITION

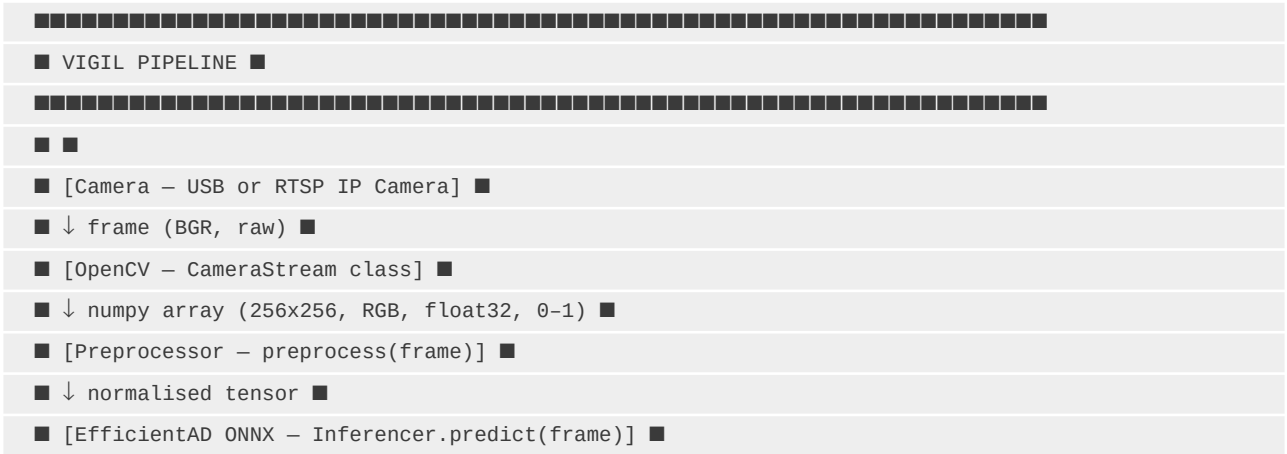
TASK	DESCRIPTION	DELIVERABLE
Demo video	A screen recording of the live pipeline: camera feed visible, a defective cashew placed in frame, alert fires, screenshot saved, dashboard updates. Minimum 60 seconds of recording.	Required by 23 Jun 6 PM
Model performance	EfficientAD trained on your collected dataset achieves at least 80% separation between good and defective cashew anomaly scores when tested on the test set.	Required
GitHub repo	All code merged into main branch. Clean folder structure. README with setup instructions. No commented-out junk code.	Required
Zero crashes	The full pipeline must run for 5 continuous minutes without crashing or hanging. Tested on Day 5.	Required

SECTION 02

System Architecture

Full pipeline from camera to dashboard. Every intern must understand the whole system.

THE PIPELINE



[illegible]

FOLDER STRUCTURE — REQUIRED

```
vigil-mvp/
├── camera/
│   ├── camera.py ← CameraStream class (Intern 1)
│   ├── preprocessor.py ← preprocess() function (Intern 1)
│   └── model/
│       ├── train.py ← EfficientAD training script (Intern 2)
│       ├── export.py ← ONNX export (Intern 2)
│       ├── inferencer.py ← Inferencer class (Intern 2)
│       ├── config.yaml ← threshold + model path (Intern 2)
│       └── alerts/ ← JPEG screenshots saved here (auto-created)
├── backend/
│   ├── main.py ← FastAPI app (Intern 3)
│   ├── alert_handler.py ← AlertHandler (Intern 3)
│   ├── logger.py ← SQLite logger (Intern 3)
│   └── dashboard/
│       ├── dashboard.html ← Single-file dashboard (Intern 3)
│       └── vigil_dataset/
│           ├── train/good/ ← 150-200 good cashew images
│           ├── test/
│           ├── good/ ← 20 good validation images
│           ├── defective/ ← 15-20 defective cashew images
│           ├── schema.sql ← DB schema (Intern 3)
│           ├── requirements.txt ← All Python deps
│           └── README.md ← Setup + run instructions
```

API ENDPOINTS

```
GET /feed → streams live camera frames as JPEG (MJPEG stream)
GET /alerts → returns last N alert records as JSON from SQLite
```

GET /status → returns system health: camera_ok, model_loaded, uptime

No other endpoints. No authentication. No POST requests in MVP.

■ These three endpoints are the only API surface. Do not add more. Do not change the endpoint names. The dashboard depends on exact URLs.

SECTION 03

Rules for Agent-Assisted Coding

You will use an AI coding assistant (Claude, Cursor, GitHub Copilot). These rules are mandatory.

R-0 1	Read before you generate Before prompting the AI for any task, read the full task description in this brief. The AI does not know this project. You must inject context into every prompt. The prompts provided in this document already include the right context — use them.
R-0 2	One task = one prompt session Do not ask the AI to write multiple files or multiple functions in one prompt. One task, one file, one prompt session. Verify the output before moving on.
R-0 3	Never accept code you do not understand If the AI generates code with a function you cannot explain line-by-line, ask it to explain that function before you commit it. You are responsible for every line in your PR. 'The AI wrote it' is not an acceptable explanation.
R-0 4	Run and verify every output Every file the AI generates must be run and tested immediately. Do not generate five files and test them all at once. Generate one, run it, fix it, commit.
R-0 5	No hallucinated libraries If the AI imports a library you have not seen before, check that it exists on PyPI before using it. Run: <code>pip show </code> . If it does not exist, reject the code.
R-0 6	No external API calls The AI may suggest calling cloud APIs, downloading pre-trained models from the internet, or using hosted services. Reject all of these. VIGIL runs fully offline. All model weights and dependencies must be local.
R-0 7	Prompts are starting points The prompts in this document are carefully written but may need minor adjustments for your specific system path or environment. Edit path names and filenames as needed. Do not change the technical requirements stated in the prompts.
R-0 8	Always add docstrings Every function the AI writes must have a one-line docstring. If the AI skips it, add this to your prompt: 'Add a one-line docstring to every function.'

**R-0
9****Commit after each task**

After each task is verified and working, commit to your branch with a clear message. Example: 'feat: add CameraStream class with RTSP reconnect logic'. Do not accumulate uncommitted work.

**R-1
0****PR before Day-end**

Submit your pull request by 5:30 PM each day. Daily review happens at 6:00 PM IST. PRs submitted after 6 PM will be reviewed the next day.

■ Violation of R-06 (external APIs) or R-05 (hallucinated libraries) will result in your PR being rejected regardless of how much else works. Check every import.

SECTION 04

Dataset Collection Protocol

All three interns complete this together on Day 1 (June 19). Do not begin any coding tasks until the dataset is complete.

EfficientAD is an unsupervised anomaly detection model. It does not require labelled defect images. It trains exclusively on images of good, normal cashew kernels. The quality of your dataset directly determines the quality of the model. A poor dataset will give you a model that cannot distinguish good from defective. Treat dataset collection as engineering work — not photography.

EQUIPMENT & MATERIALS

TASK	DESCRIPTION	DELIVERABLE
Cashew kernels	Minimum 500g. Mixed sizes are fine. Source from a local shop. Do not use salted, roasted, or seasoned cashews — raw only.	Raw material
Backdrop	One sheet of white A4 paper or white cardboard on a flat surface. Replace the backdrop if it gets dirty or creased.	White, clean, flat
Camera	Phone camera preferred. Fix the phone at a consistent height using a stack of books or a phone stand. Do not handhold.	Fixed position only
Lighting	Single light source only. Window daylight with no direct sunlight, or a desk lamp pointed at the backdrop. No overhead tube lights — they cause harsh shadows.	Consistent, shadowless
Target count	200 training images (good kernels) + 20 good validation images + 15 defective validation images = 235 total images.	Non-negotiable minimum

PHOTOGRAPHY PROTOCOL

Place one kernel in the centre of the backdrop. The kernel should fill roughly 50–60% of the frame. Take the photo. Move the kernel to a slightly different orientation or position, take the next photo. Repeat. Do not use the same kernel in the same position twice in a row — vary orientation. Discard any photo where the kernel is blurred, shadowed, or partially off-frame.

WHAT COUNTS AS A GOOD KERNEL

TASK	DESCRIPTION	DELIVERABLE
Include	Whole, intact kernel. Uniform light beige or cream colour. No cracks, chips, or missing pieces. Minor size variation is fine.	Train set: 200 images
Exclude	Broken or chipped kernels, discoloured patches (brown, black, grey), shrivelled or deformed kernels, any kernel with visible mould or fungal spots.	Never in train set
Defective set	Collect 15–20 defective kernels separately after the good set is done. Use broken, chipped, discoloured, or shrivelled kernels. Photograph each defective kernel the same way as the good set.	Test set only — never training

FILE NAMING & FOLDER STRUCTURE

```
vigil_dataset/  
train/  
good/  
000.jpg ← first good kernel image  
001.jpg  
002.jpg  
... ← continue to at least 199.jpg (200 images)  
test/  
good/  
000.jpg ← 20 good validation images (separate from train set)  
... ← 019.jpg  
defective/  
000.jpg ← defective kernel images  
... ← 014.jpg minimum (15 images)
```

Rename images sequentially using any file manager or a simple Python rename script. Do not use camera-generated filenames like `IMG_20260619_093421.jpg`.

RENAME SCRIPT — use this if needed

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python script `rename_images.py` that:

1. Accepts a folder path as a command-line argument (`sys.argv[1]`)
2. Finds all `.jpg` and `.jpeg` files in that folder
3. Sorts them by creation time (`os.path.getctime`)
4. Renames them to `000.jpg`, `001.jpg`, `002.jpg` ... in order
5. Prints each rename operation as: `'old_name -> new_name'`

Use `os`, `sys`, and `pathlib` only. No third-party libraries.

Add a one-line docstring to every function.

INTERN 1

Camera Layer & Frame Preprocessing

Python 3.10 · OpenCV (cv2) · numpy

SPRINT DAYS

Day
2–3

You are responsible for everything between the physical camera and the inference engine. Your module is the entry point of the entire pipeline. If your output frames are the wrong size, wrong dtype, or wrong value range, the model will produce meaningless results. Precision matters here.

ENVIRONMENT SETUP — Day 1 (after dataset collection)

```
pip install opencv-python numpy
python -c "import cv2; import numpy; print('OK')"
```

```
# Verify camera is detected:
python -c "import cv2; cap = cv2.VideoCapture(0); print(cap.read()[0])"
```

```
# Should print: True
```

TASK 1.1 — CameraStream Class

Write a Python class `CameraStream` in `camera/camera.py`. The class must accept either a USB device index (integer, e.g. 0 for the first camera) or an RTSP URL string (e.g. `'rtsp://192.168.1.10:554/stream'`). It must expose a `read()` method that returns the next frame as a numpy array in BGR format, and a `release()` method that closes the stream. If the stream disconnects, it must automatically retry every 5 seconds and log the disconnect event with a timestamp to stdout.

TECHNICAL REQUIREMENTS

TASK	DESCRIPTION	DELIVERABLE
Constructor	<code>__init__(self, source)</code> where <code>source</code> is <code>int</code> or <code>str</code> . Open the capture immediately in <code>__init__</code> . Raise <code>ValueError</code> if the source cannot be opened.	<code>camera.py</code>
<code>read()</code> method	Returns (success: bool, frame: <code>np.ndarray</code>). Frame is BGR. If frame is <code>None</code> (stream dropped), trigger reconnect logic and return (<code>False, None</code>) for that call.	<code>camera.py</code>
Reconnect logic	If <code>read()</code> returns <code>None</code> , sleep 5 seconds, attempt to reopen the source. Log: <code>'[TIMESTAMP] Camera disconnected. Retrying in 5s...'</code> Keep retrying until reconnected. Log reconnect: <code>'[TIMESTAMP] Reconnected.'</code>	<code>camera.py</code>
<code>release()</code> method	Calls <code>self.cap.release()</code> . Called on system shutdown.	<code>camera.py</code>

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

I am building a real-time camera ingestion module for an industrial vision system.

Write a Python class called `CameraStream` in a file called `camera.py`.

Requirements:

- The class lives in `camera/camera.py`
- `__init__(self, source)` accepts either an int (USB device index) or a str (RTSP URL)
- It opens a `cv2.VideoCapture` immediately in `__init__`
- It raises `ValueError` if the source cannot be opened
- `read(self)` returns a tuple: (bool, np.ndarray) – same as `cv2.VideoCapture.read()`
- If the frame is `None` (stream dropped), the method logs the disconnect with a timestamp using Python's `datetime` module, sleeps 5 seconds, and attempts to reopen the capture. It returns (False, None) for the failed call.
- Reconnect attempts are infinite until success. Log reconnect success with timestamp.
- `release(self)` calls `self.cap.release()`
- Every method must have a one-line docstring
- Import only: `cv2`, `numpy`, `time`, `datetime`
- No external libraries. No threading. No queues in this version.

TASK 1.2 — Frame Preprocessor

Write a `preprocess(frame)` function in `camera/preprocessor.py`. This function takes a raw BGR numpy array from the camera and returns a normalised float32 numpy array ready for EfficientAD inference. The exact output specification is non-negotiable — the model expects precisely this format.

EXACT OUTPUT SPECIFICATION

Input: np.ndarray shape=(H, W, 3) dtype=uint8 range=[0, 255] colour=BGR

Output: np.ndarray shape=(256, 256, 3) dtype=float32 range=[0.0, 1.0] colour=RGB

Steps (in this exact order):

1. `cv2.resize(frame, (256, 256))` ← resize to 256x256
2. `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` ← convert BGR to RGB
3. `frame.astype(np.float32) / 255.0` ← normalise to [0, 1]
4. `return frame`

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python function called `preprocess` in `camera/preprocessor.py`.

Function signature: `preprocess(frame: np.ndarray) -> np.ndarray`

Processing steps (in this exact order):

1. Resize the input frame to 256x256 using `cv2.resize`
2. Convert colour space from BGR to RGB using `cv2.cvtColor` with `cv2.COLOR_BGR2RGB`
3. Convert dtype to float32 and divide by 255.0 to normalise pixel values to [0.0, 1.0]
4. Return the processed frame

Add a one-line docstring.

Input: `np.ndarray` of shape (H, W, 3), dtype `uint8`, BGR colour

Output: `np.ndarray` of shape (256, 256, 3), dtype `float32`, RGB, values in [0.0, 1.0]

Import only: `cv2`, `numpy`. No other libraries.

TASK 1.3 — Test Script

Write a test script `camera/test_camera.py` that validates both `CameraStream` and `preprocess()` work correctly end-to-end. This is your verification that your module is ready for integration.

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python test script called `test_camera.py` in the `camera/` folder.

The script should:

1. Import `CameraStream` from `camera.camera` and `preprocess` from `camera.preprocessor`
2. Open `CameraStream` with `source=0` (USB camera index)
3. Read exactly 10 frames in a loop
4. Pass each frame through `preprocess()`
5. For each processed frame, assert:
 - `frame.shape == (256, 256, 3)`
 - `frame.dtype == np.float32`
 - `frame.min() >= 0.0` and `frame.max() <= 1.0`
6. Print for each frame: `'Frame N: shape={shape} dtype={dtype} min={min:.3f} max={max:.3f}'`
7. Call `stream.release()` at the end
8. If all assertions pass, print: `'ALL CHECKS PASSED'`

Import only: `cv2`, `numpy`, `sys`. Add a comment explaining each step.

■ **Day 3 deadline:** `test_camera.py` must print **ALL CHECKS PASSED** before you submit your PR. Do not submit until this passes cleanly.

INTERN 2

ML Pipeline — EfficientAD Training, ONNX Export & Inference

Python 3.10 · anomalib · ONNX Runtime · numpy

SPRINT DAYS

Day
2–4

You are responsible for the brain of VIGIL — the anomaly detection model. Your deliverable is a fully trained EfficientAD model exported to ONNX format, and an Inferencer class that accepts a preprocessed frame and returns an anomaly score. This is the most technically demanding module. Follow the steps precisely.

ENVIRONMENT SETUP — Day 2 Morning

```
pip install anomalib
pip install onnx onnxruntime

# Verify anomalib installed correctly:
python -c "from anomalib.models import EfficientAd; print('anomalib OK')"
```

```
# Verify ONNX runtime:
python -c "import onnxruntime as ort; print(ort.get_device())"
```

```
# Should print: CPU
```

BACKGROUND: WHAT IS EFFICIENTAD

EfficientAD is a lightweight teacher-student anomaly detection architecture. A pre-trained teacher network generates feature representations of normal images. A smaller student network learns to mimic those representations. At inference time, regions where the student fails to mimic the teacher indicate anomalies — because the student has never seen those patterns. The output is a pixel-level anomaly map, which is aggregated into a single image-level anomaly score. Scores near 0 mean normal; scores near 1 mean anomalous.

TASK 2.1 — Dataset Configuration

Create the anomalib dataset configuration file. anomalib uses YAML config files to define the dataset location and structure. The Folder datamodule expects your dataset in the exact folder structure defined in Section 04.

REQUIRED CONFIG FILE: model/dataset_config.yaml

```
dataset:
  name: vigil_cashew
  format: folder
  path: ../vigil_dataset # relative to model/ directory
  normal_dir: train/good
  abnormal_dir: test/defective
  normal_test_dir: test/good
  task: classification # image-level scoring, not pixel segmentation
  image_size: 256
  train_batch_size: 32
  eval_batch_size: 32
  num_workers: 0 # important: set to 0 on Windows
```

Create this file exactly as shown. Adjust the path if needed for your OS.

TASK 2.2 — Training Script

Write the EfficientAD training script. Training on 200 images on CPU takes approximately 20–40 minutes. Run it overnight on Day 2 if needed, but start it by 4 PM so you can verify it launched correctly.

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

I am training an EfficientAD anomaly detection model using the anomalib library.

Write a Python training script called train.py in the model/ folder.

Requirements:

- Use anomalib's EfficientAd model
- Load dataset using anomalib's Folder datamodule
- Dataset path: '../vigil_dataset' (relative to model/ folder)
- normal_dir: 'train/good'
- abnormal_dir: 'test/defective'
- normal_test_dir: 'test/good'
- Task: 'classification' (image-level, not segmentation)
- Image size: 256x256
- Use PyTorch Lightning Trainer with max_epochs=100
- Accelerator: 'cpu' (no GPU)
- Save the best model checkpoint to model/checkpoints/
- Print training progress to console
- At the end of training, print the image-level AUROC score on the test set
- Add a docstring to every function
- Import only from: anomalib, pytorch_lightning, torch, pathlib, yaml

Use anomalib version compatible with: from anomalib.models import EfficientAd

Use anomalib's Engine class if available, otherwise use Trainer directly.

■ If anomalib API has changed and the prompt does not work directly, ask the AI: 'What is the correct anomalib API for training EfficientAD in anomalib version X?' where X is the version from pip show anomalib. Do not guess API names.

TASK 2.3 — ONNX Export

After training completes and you have a checkpoint, export the model to ONNX format. ONNX allows the model to run without PyTorch installed and is faster for inference. The exported file must load and run correctly before you proceed.

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

I have a trained anomalib EfficientAD model checkpoint at model/checkpoints/.

Write a Python script called export.py in the model/ folder.

The script should:

1. Load the trained EfficientAD model from the best checkpoint in model/checkpoints/
2. Export the model to ONNX format using anomalib's export functionality
OR using torch.onnx.export if anomalib export is not available
3. Save the ONNX file to model/vigil_efficientad.onnx
4. After saving, load the ONNX file using onnxruntime.InferenceSession and run a test inference with a dummy numpy input of shape (1, 3, 256, 256) dtype float32
5. Print: 'ONNX export verified. Output shape: {shape}'
6. If verification fails, raise RuntimeError with a descriptive message

Add a one-line docstring to every function.

Import only: torch, anomalib, onnxruntime, numpy, pathlib

TASK 2.4 — Inferencer Class

Write the Inferencer class that loads the ONNX model and runs predictions. This class will be imported by the FastAPI backend. The output must be a single float between 0 and 1.

EXACT INTERFACE REQUIRED — DO NOT DEVIATE

```
class Inferencer:
    def __init__(self, onnx_path: str, threshold: float = 0.5):
        # loads the ONNX model
        ...

    def predict(self, frame: np.ndarray) -> float:
        # frame: shape (256, 256, 3), dtype float32, values [0, 1], RGB
        # returns: anomaly score as float in [0.0, 1.0]
        ...

    def is_defect(self, frame: np.ndarray) -> tuple[bool, float]:
        # returns (True, score) if score > threshold, else (False, score)
        ...
```


AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python class called `Inferencer` in `model/inferencer.py`.

Class interface:

- `__init__(self, onnx_path: str, threshold: float = 0.5)`
- Loads the ONNX model from `onnx_path` using `onnxruntime.InferenceSession`
- Stores the threshold
- Prints: `'Model loaded from {onnx_path}'`
- `predict(self, frame: np.ndarray) -> float`
- Input: frame with shape (256, 256, 3), dtype float32, values [0.0, 1.0], RGB
- Preprocessing: transpose to (1, 3, 256, 256) for ONNX input (add batch dim)
- Run ONNX inference
- Extract the anomaly score (a single float) from the model output
- Normalise the score to [0.0, 1.0] using min-max normalisation if needed
- Return the score as a Python float
- `is_defect(self, frame: np.ndarray) -> tuple[bool, float]`
- Calls `self.predict(frame)`
- Returns (True, score) if score > self.threshold
- Returns (False, score) otherwise

Add a one-line docstring to every method.

Import only: `onnxruntime`, `numpy`. No `torch`, no `anomalib` in this file.

TASK 2.5 — config.yaml

Create `model/config.yaml` with the model path and threshold.

```
model:
  onnx_path: model/vigil_efficientad.onnx
  threshold: 0.5 # adjust after reviewing test set scores
  # if good scores cluster below 0.3, set threshold to 0.35
  # if defective scores cluster above 0.7, set threshold to 0.6
```

After training, run your `Inferencer` on all test/good and test/defective images. Print the scores. Set the threshold to a value that sits clearly between the two distributions. Document your chosen threshold in your PR description.

INTERN 3

Backend, Alert System & Dashboard*Python 3.10 · FastAPI · SQLite3 · HTML / CSS / JS*

SPRINT DAYS

**Day
2–5**

You are responsible for the system backbone and user interface. Your FastAPI server ties all modules together. Your dashboard is what will be visible in the demo video. Every component you write must be clean, fast, and crash-resistant. The dashboard must look professional — it represents the product to a potential client.

ENVIRONMENT SETUP — Day 2 Morning

```
pip install fastapi uvicorn python-multipart
python -c "import fastapi; import uvicorn; print('FastAPI OK')"

# SQLite3 is part of Python stdlib – no install needed
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

TASK 3.1 — SQLite Schema & Logger

Create the database schema and the logger module. Every alert fired by the system is written to SQLite. The dashboard reads from this database. Get this right first — everything else depends on it.

SCHEMA — schema.sql

```
CREATE TABLE IF NOT EXISTS alerts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp TEXT NOT NULL, -- ISO format: 2026-06-20T14:32:05
  anomaly_score REAL NOT NULL, -- float, 0.0 to 1.0
  screenshot_path TEXT NOT NULL -- relative path: alerts/20260620_143205.jpg
);
```

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python module called `logger.py` in `backend/`.

It must contain:

```
1. init_db(db_path: str = 'vigil.db') -> None
- Opens a SQLite connection to db_path
- Creates the alerts table if it does not exist:
id INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT NOT NULL,
anomaly_score REAL NOT NULL,
screenshot_path TEXT NOT NULL
- Commits and closes the connection

2. log_alert(timestamp: str, score: float, screenshot_path: str,
db_path: str = 'vigil.db') -> int
- Inserts a row into alerts
- Returns the inserted row id

3. get_recent(n: int = 20, db_path: str = 'vigil.db') -> list[dict]
- Returns the last n alerts as a list of dicts:
[{'id':..., 'timestamp':..., 'anomaly_score':..., 'screenshot_path':...}]
- Ordered by id DESC (most recent first)
```

Add a one-line docstring to every function.

Import only: `sqlite3`, `pathlib`. No third-party libraries.

TASK 3.2 — Alert Handler

The alert handler is called every time the anomaly score exceeds the threshold. It saves a JPEG screenshot of the defective frame and returns the alert record.

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a Python module called `alert_handler.py` in `backend/`.

It must contain one function:

```
handle_alert(frame: np.ndarray, score: float,
             alerts_dir: str = 'alerts') -> dict
```

The function should:

1. Create the `alerts_dir` directory if it does not exist (`pathlib.Path.mkdir`)
2. Generate a filename from the current timestamp:

format: `YYYYMMDD_HHMMSS.jpg` example: `20260620_143205.jpg`

3. Convert the frame from RGB float32 `[0,1]` back to BGR uint8 `[0,255]`
(`frame * 255`).`astype(np.uint8)` then `cv2.cvtColor` to BGR

4. Save the frame as a JPEG using `cv2.imwrite` to `alerts_dir/filename`

5. Build an alert record dict:

```
{'timestamp': iso_timestamp, 'anomaly_score': score,
 'screenshot_path': f'{alerts_dir}/{filename}'}
```

6. Return the alert record dict

Add a one-line docstring.

Import only: `cv2`, `numpy`, `pathlib`, `datetime`. No third-party libraries.

TASK 3.3 — FastAPI Backend

Write the FastAPI application that wires all modules together and exposes the three API endpoints. This is the central integration point of the entire system.

HOW THE MAIN LOOP WORKS

On startup:

1. `init_db()` ← initialise SQLite
2. `CameraStream(source)` ← open camera
3. `Inferencer(onnx_path)` ← load model

Per frame (background thread, 10 FPS max):

4. `stream.read()` ← get frame
5. `preprocess(frame)` ← normalise
6. `inferencer.is_defect(frame)` ← get (`is_defect`, `score`)
7. If `is_defect`:

```
    alert = handle_alert(frame, score)
```

```
    log_alert(**alert)
```

8. Store latest frame for `/feed` endpoint

Endpoints serve from shared state (latest frame, SQLite DB)

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a FastAPI application in backend/main.py for an industrial vision inspection system.

Imports available (all already written):

```
from camera.camera import CameraStream
from camera.preprocessor import preprocess
from model.inferencer import Inferencer
from backend.alert_handler import handle_alert
from backend.logger import init_db, log_alert, get_recent
```

Global state (module-level variables):

```
stream = None # CameraStream instance
inferencer = None # Inferencer instance
latest_frame = None # most recent raw BGR frame (numpy array)
start_time = None # datetime.now() at startup
```

Startup event (lifespan or @app.on_event('startup')):

- init_db()
- Create CameraStream(source=0)
- Create Inferencer('model/vigil_efficientad.onnx')
- Start a background thread that runs the inference loop at 10 FPS max
- Store start_time = datetime.now()

Inference loop (runs in background thread, daemon=True):

- While True: read frame, preprocess, run is_defect
- Update latest_frame with the raw BGR frame
- If is_defect: call handle_alert, then log_alert
- Sleep 0.1 seconds (10 FPS)

Endpoints:

GET /feed:

- Returns latest_frame as a JPEG MJPEG stream
- Use StreamingResponse with media_type='multipart/x-mixed-replace; boundary=frame'
- Generator: encode latest_frame with cv2.imencode('.jpg', frame)
- Yield each frame as: b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + jpg_bytes + b'\r\n'

GET /alerts:

- Query param: n: int = 20
- Returns JSONResponse with get_recent(n) result

GET /status:

- Returns JSONResponse:

```
{'camera_ok': stream is not None, 'model_loaded': inferencer is not None,
'uptime_seconds': (datetime.now() - start_time).seconds}
```

Add a one-line docstring to every function.

Import only: fastapi, cv2, numpy, threading, time, datetime, and the local modules above.

TASK 3.4 — HTML Dashboard

Build a single-file HTML dashboard. No frameworks. No React. No Tailwind. No Node. Pure HTML, CSS, and JavaScript. This file is what will be visible in the demo video. It must look professional and update live without page reloads.

LAYOUT SPECIFICATION

[illegible]

AGENT PROMPT — COPY & PASTE THIS INTO YOUR AI ASSISTANT

Write a single HTML file called `dashboard.html` in the `dashboard/` folder.

No external CSS frameworks. No JavaScript frameworks. No CDN links.

Pure HTML5, CSS3, and vanilla JavaScript only.

Layout (two-column, full viewport height):

- Left column (60% width): title bar + live MJPEG camera feed
 - Title: 'VIGIL · Live Quality Inspection'
 - Camera feed: `<img id='feed' src='http://localhost:8000/feed' width='100%'>`
 - Status indicator: green dot + 'ONLINE' text (static for MVP)
 - Right column (40% width): 'Recent Alerts' heading + scrollable table
 - Table columns: Timestamp | Score | Thumbnail
 - Thumbnail: small `<img>` tag using the `screenshot_path` field
 - Score: display as a percentage with 1 decimal place, e.g. '87.3%'
 - Colour the score cell: green if `< 60%`, orange if `60-80%`, red if `> 80%`
- JavaScript (no frameworks):
- On page load and every 3 seconds, fetch `http://localhost:8000/alerts?n=20`
 - Parse the JSON response and re-render the alerts table
 - Use `setInterval(fetchAlerts, 3000)`
 - Handle fetch errors: if the server is unreachable, show 'Connection lost' in red

Styling requirements:

- Dark background (#0D0D0D), white or light grey text
- Monospaced font for scores and timestamps (font-family: monospace)
- No hover effects on feed image
- Table rows: alternating subtle background colours
- The dashboard must look clean and professional – it will be shown to clients

■ The dashboard fetches from `http://localhost:8000` — this is hardcoded for the local demo. Do not add any proxy, CORS workaround, or environment variable system. Simple and direct.

SECTION 08

5-Day Sprint Timeline

June 19–23, 2026. All deadlines are at 5:30 PM IST unless stated otherwise.

DATE	DAY	INTERN 1	INTERN 2	INTERN 3
19 Jun	Day 1 Setup & Dataset	Dataset collection (all interns together — morning) Afternoon: env setup, install OpenCV, test cv2 import	Dataset collection (morning) Afternoon: install anomalib, onnxruntime, verify imports	Dataset collection (morning) Afternoon: install FastAPI, uvicorn, run hello world
20 Jun	Day 2 Core Modules	Write CameraStream class RTSP + USB support Reconnect logic	Create dataset_config.yaml Write train.py Launch training (let run overnight)	Write schema.sql Write logger.py (init_db, log_alert, get_recent) Unit test logger
21 Jun	Day 3 Completion + Test	Write preprocess() function Write test_camera.py All checks must pass PR by 5:30 PM	Verify training completed Check AUROC score Write export.py Run ONNX export	Write alert_handler.py Manual test: pass dummy frame, verify JPEG saved PR by 5:30 PM
22 Jun	Day 4 Inference + Backend	Support integration Fix any bugs in camera module Help test end-to-end	Write inferencer.py Set threshold from test scores Update config.yaml PR by 5:30 PM	Write main.py (FastAPI) Wire all modules Test /feed /alerts /status Write dashboard.html
23 Jun	Day 5 Integration + Demo	Full pipeline end-to-end test All three interns together Demo video recording Final README	Full pipeline end-to-end test Fix inference bugs Demo video recording	Full pipeline end-to-end test Dashboard final check Demo video recording

DAILY REVIEW PROCESS — 6:00 PM IST every day

TASK	DESCRIPTION	DELIVERABLE
PR submission	All code for the day must be in a pull request by 5:30 PM. No PR = no review = that day's work is not counted.	Hard cutoff
Review call	15-minute call at 6:00 PM. Each intern presents: what I built, show it running, what broke, what's tomorrow.	Mandatory
Blocker protocol	If you are blocked for more than 90 minutes on a single issue, do not wait until the review call. Message the group immediately.	Same-day resolution

SECTION 09

Day 5 — Integration & Demo Protocol

June 23. All three interns work together. This day is exclusively integration and recording.

INTEGRATION SEQUENCE — follow in this exact order

1. Merge all PRs

All branches merged to main before 10:00 AM. Resolve conflicts in the morning.

2. Install all dependencies

From the root of the repo: `pip install -r requirements.txt`. Verify no import errors.

3. Initialise the database

`python -c "from backend.logger import init_db; init_db()"` — confirm `vigil.db` created.

4. Launch the FastAPI server

`cd vigil-mvp && uvicorn backend.main:app --host 0.0.0.0 --port 8000` — verify all three startup logs appear: DB initialised, camera opened, model loaded.

5. Open the dashboard

Open `dashboard/dashboard.html` in a browser. Verify the live feed is visible and the alerts table loads.

6. Run the good-cashew test

Place 5 good cashew kernels in front of the camera one by one. Each should NOT trigger an alert. Verify in the dashboard.

7. Run the defect test

Place a broken/dicoloured cashew kernel in front of the camera. An alert must fire within 3 seconds. Verify screenshot saved to `alerts/` and row appears in the dashboard table.

8. Run for 5 minutes continuously

Let the system run with a mix of good and defective kernels for 5 minutes. No crashes, no hangs, dashboard auto-refreshes every 3 seconds.

9. Record the demo video

Screen record the full pipeline: dashboard visible, camera feed visible, defective kernel placed, alert fires, screenshot thumbnail appears in table. Minimum 60 seconds. Export as MP4.

10. Push final code and README

All code in main branch. README includes: prerequisites, install steps, how to collect dataset, how to train, how to run the server. Commit message: 'final: VIGIL MVP v1.0 — demo ready'

The demo video is the final deliverable. One working system. One camera. One production line simulation. A defective cashew flagged on screen in real time. That video is the asset that moves this project from intern sprint to client pilot.

VIGIL MVP · INTERN SPRINT · JUN 19–23 2026